

LearnSync Database and Frontend Schema Analysis

Applied connection

The teacher grading sheet now loads records for a selected ``class_id`` through ``GET /api/grades/class/{class_id}``. The subject workspace links to that URL, and the final-grade link preserves the selected class. The existing grading tabs and four data-column table layout were kept.

Matched data paths

Frontend surface	Database source	Result
Subject title and class context	<code>`class.class_id`, `class.subject`, `class.class_code`</code>	Connected through the subject <code>`class_id`</code> URL.
Sections	<code>`section.class_id`</code>	Returned with the selected class.
Student names	<code>`enrollment -> student -> account`</code>	Returned only for students enrolled in the selected class.
Attendance tab	<code>`attendance.class_id`, `student_id`, `score`, `is_present`, `date_created`</code>	Returned and grouped by record date.
Quiz tab	<code>`quiz.class_id` and `quiz_score.quiz_id/student_id/total score`</code>	Returned by quiz and student.
Activities tab	<code>`activity.class_id` and latest `act_submission` score</code>	Returned by activity and student.
Grading weights	<code>`grading_policy.class_id`</code>	Returned with the class.

Remaining schema mismatches

1. Grading periods are not represented consistently

The frontend has ``Midterm`` and ``Final`` filters and the final-grade table has separate midterm/final columns. ``attendance``, ``quiz_score``, and ``act_submission`` have no term or grading-period column. The generic ``grade.type`` column exists, but it is not linked to individual attendance, quiz, or activity records and is currently unused by the backend.

****Effect:**** the selected term cannot safely filter raw records or produce separate midterm/final results.

****Recommended schema change:**** add a controlled ``grading_period`` column, such as ``Midterm`/`Final``, to raw grading records or create a normalized grading-record table with ``class_id``, ``student_id``, ``period``, ``category``, ``item_id``, ``score``, and ``total``.

2. Quiz totals are missing

The frontend table displays a total above each quiz column. ``quiz_score`` stores only ``total_score``; ``quiz`` has no ``total_items`` or maximum score. The connected UI therefore displays ``-`` for quiz totals instead of inventing a maximum.

****Recommended schema change:**** add ``max_score`` or ``total_items`` to ``quiz`` and use it when rendering and calculating percentages.

3. Attendance item totals are not stored

``attendance`` stores ``is_present``, ``score``, and a date per row, but there is no attendance-item/header table containing the total possible value or label. The connected UI uses the record date as the column label and displays ``-`` for its total.

****Recommended schema change:**** add an ``attendance_item`` table with ``class_id``, ``label``, ``record_date``, and ``total``, then reference it from attendance rows.

4. Activities and submissions are separate from generic grades

Activity scores are stored on ``act_submission``, while the frontend's activity sheet treats each activity as a score column. There is no explicit score record for an enrolled student who has not submitted, and ``activity.points`` is the only available maximum.

****Effect:**** missing submissions correctly render as blank, but absence, zero, and ungraded are not distinguishable in the sheet.

****Recommended schema change:**** add a per-student activity-grade/status record or define explicit grading states for ``Missing``, ``Ungraded``, and ``Scored``.

5. Final-grade publication is frontend-only

The final-grade page includes a Published/Not Published selector, but no publication field or endpoint exists in the database/backend. Its computed values were previously hard-coded and are not yet persisted.

****Recommended schema change:**** add a class/period grade publication record and a final-grade calculation endpoint backed by the raw records.

6. Class/section integrity is incomplete

Several existing subject endpoints find a section by section code alone rather than by both ``class_id`` and section code. Section codes such as ``3A`` can repeat in different classes, so this can attach records to the wrong class.

****Recommended schema change:**** enforce and query section ownership with ``(class_id, section)``, plus a unique constraint on that pair.

7. Foreign-key constraints are mostly absent

The SQL defines primary keys and some uniqueness constraints, but the core relationships between classes, sections, enrollments, grades, quizzes, activities, and students are not consistently enforced with foreign keys.

****Effect:**** orphaned records can be created and the frontend must rely on application code for referential integrity.

****Recommended schema change:**** add foreign keys with deliberate delete behavior, then add indexes for the class/student lookup paths used by grading.

Files changed

- ``backend/routers/grades.py``
- ``backend/main.py``
- ``backend/routers/classes.py``
- ``static/teacher/grading.html``
- ``static/teacher/subject.html``